

# 모델링 설명

## 수리모형 - 1단계: 대덕구 최소 설치대수 (가중 커버 목표형)

대덕구 내 수요지점이 전체의 90% 이상( $\alpha=0.9$ ) 커버 혹은 서비스가 되도록 보장하면서, 최소한의 무인택배함을 설치하기 위한 최소 설치대수( $p^*$ )를 산정하였다.

이때 이러한 수요지점을 커버하는 것을 보장하는 거리인 커버 임계거리( $T=1.5$  km) 이내에 포함되면 무인택배함 서비스가 가능한 것으로 가정하였다.

\*  $\alpha$ (목표 커버율)과  $T$ (커버 임계거리)는 정책 목표나 분석 조건 등에 따라 자유롭게 설정할 수 있다.

수리모형의 파라미터와 목적함수, 제약식은 다음과 같다.

### 정의

Sets and parameters:

- $I$ : set of demand points
- $J$ : set of candidate facility points
- $d_{ij}$ : distance between demand point  $i$  and candidate facility point  $j$
- $R$ : coverage radius (in meters)
- $a_{ij}$ : Coverage indicator. 1 if  $d_{ij} \leq R$ , 0 otherwise, where  $a_{ij} \in \{0, 1\}$
- $w_i$ : Demand weight of grid  $i$
- $W_{tot}$ : Total weighted Demand
- $\tau$ : Target coverage ratio, where  $\tau \in (0, 1]$

Decision variables:

- $x_j$ : 1 if facility  $j$  is selected, 0 otherwise, where  $x_j \in (0, 1)$
- $z_i$ : 1 if demand point  $i$  is covered (within radius  $R$ ), 0 otherwise, where  $z_i \in (0, 1)$

### 수리모델

$$\min \sum_{j \in J} w_i d_{ij} y_{ij} \quad (1)$$

$$\text{s.t.} \quad \sum_{j \in J} a_{ij} x_j \geq z_i \quad \forall i \in I \quad (2)$$

$$\sum_{i \in I} w_i z_i \geq \tau W_{tot} \quad (3)$$

$$x_j \in \{0, 1\}, z_i \in \{0, 1\} \quad (4)$$

- **(1)** 목적함수는 전체 수요지점과 후보지 간의 가중 거리합을 최소화한다.
- **(2)** 각 수요지점은 커버 반경  $R$  이내에 하나 이상의 후보지가 선택될 경우 커버된 것으로 간주한다.
- **(3)** 전체 수요 가중치 중 최소  $\tau$  이상이 커버되도록 제한한다.

- **(4)** x, z는 각각 이진변수이다.

## 코드 구현 - 1단계

```
# -*- coding: utf-8 -*-
import geopandas as gpd
import pandas as pd, numpy as np
from shapely.ops import unary_union
import gurobipy as gp
from gurobipy import GRB

daejeok_grid_shp = "대구광역시_대덕구.shp"
transit_csv = "preprocessing ver7.csv"
cands_csv = "finaldestination.csv"

# ----- 입력 -----
R_COVER_M = 1500 # 커버 반경 (m)
TARGET_COVER = 0.90 # 목표 커버리지(가중치 기준 90%)

POP_W, TRANSIT_W = 0.7, 0.3 # w_i 가중치

# ----- 1) 데이터 준비: 대덕구 격자/가중치 w_i -----
gdf = gpd.read_file(daejeok_grid_shp)
gdf = gdf[gdf.geometry.notnull() & gdf.geometry.is_valid].copy()
if (not gdf.crs) or (not gdf.crs.is_projected):
    gdf = gdf.to_crs(5179)

pop_col = "val"
gdf[pop_col] = pd.to_numeric(gdf[pop_col], errors="coerce").fillna(0)

df_pts = pd.read_csv(transit_csv)
df_pts = df_pts[df_pts["시군구명"].astype(str).str.contains("대덕구", na=False)].copy()
gdf_pts = gpd.GeoDataFrame(df_pts,
                           geometry=gpd.points_from_xy(df_pts["경도"], df_pts["위도"]),
                           crs=4326).to_crs(5179)

joined = gpd.sjoin(gdf_pts, gdf[["geometry"]], how="left", predicate="intersects")
joined["월 평균 승하차 인원"] = pd.to_numeric(joined["월 평균 승하차 인원"], errors="coerce").fillna(0)
bus_mean = joined[joined["시설 종류"]=="버스"].groupby("index_right")["월 평균 승하차 인원"].mean()
sub_mean = joined[joined["시설 종류"]=="지하철"].groupby("index_right")["월 평균 승하차 인원"].mean()

gdf["bus_mean"] = gdf.index.map(bus_mean).fillna(0)
gdf["sub_mean"] = gdf.index.map(sub_mean).fillna(0)
```

```

gdf["transit"] = gdf["bus_mean"] + gdf["sub_mean"]

# Min-Max → w_i
def minmax(s):
    s = s.astype(float)
    lo, hi = s.min(), s.max()
    return pd.Series(0.0, s.index) if hi-lo < 1e-12 else (s-lo)/(hi-lo)

gdf["pop_n"] = minmax(gdf["pop_col"])
gdf["tr_n"] = minmax(gdf["transit"])
gdf["w"] = POP_W*gdf["pop_n"] + TRANSIT_W*gdf["tr_n"]

# 수요점은 격자 중심
gdf_cent = gdf.copy()
gdf_cent["geometry"] = gdf_cent.geometry.centroid
I = list(range(len(gdf_cent)))

# ----- 2) 후보지 준비 -----
df_c = pd.read_csv(cands_csv, encoding="utf-8-sig")
gdf_c = gpd.GeoDataFrame(df_c,
    geometry=gpd.points_from_xy(df_c["경도"], df_c["위도"]),
    crs=4326).to_crs(5179)
J = list(range(len(gdf_c)))

# ----- 3) 커버 행렬 a_ij (within R_COVER_M) -----
ci = np.array([(p.x, p.y) for p in gdf_cent.geometry])
cj = np.array([(p.x, p.y) for p in gdf_c.geometry])
dmat = np.sqrt(((ci[:,None,:]-cj[None,:,:])**2).sum(axis=2)) # ||x||J
A = (dmat <= R_COVER_M).astype(int) # a_ij

# ----- 4) 최소 p로 목표 커버리지 달성 (maximize covered, then binary search p or minimize Sum x) -----
# 여기서는 '최소 p'를 직접 최적화: Minimize Sum x, with weighted coverage constraint
Wtot = float(gdf_cent["w"].sum())

m = gp.Model("min_p_for_target_coverage")
x = m.addVars(J, vtype=GRB.BINARY, name="x") # 후보 설치 여부
z = m.addVars(I, vtype=GRB.BINARY, name="z") # 수요 i가 커버되면 1

# 커버 논리: sum_j a_ij x_j >= z_i
for i in I:
    m.addConstr(gp.quicksum(A[i,j]*x[j] for j in J) >= z[i])

# 목표 커버리지 (가중치 기준)
m.addConstr(gp.quicksum(gdf_cent.loc[i,"w"] * z[i] for i in I) >= TARGET_COVER * Wtot)

# 목표: 설치 개수 최소화
m.setObjective(gp.quicksum(x[j] for j in J), GRB.MINIMIZE)

```

```

m.Params.TimeLimit = 300
m.optimize()

open_sites = [j for j in J if x[j].X > 0.5]
p_min = len(open_sites)
covered_weight = sum(gdf_cent.loc[i,"w"] for i in I if z[i].X > 0.5)
print(f"[RESULT] 최소 설치 개수 p = {p_min} (가중 커버리지 = {covered_weight/Wtot:.1%}, R={R_C
OVER_M}m)")

```

## 입력 및 기본 설정

- `daejeok_grid_shp`, `transit_csv`, `cands_csv` : 대덕구 인구격자, 교통 포인트, 설치 후보지 파일 경로를 의미한다.
- `R_COVER_M = 1500` : 커버 임계거리, 즉 무인택배함이 수요지점(즉, 주민 또는 서비스 대상 지역)을 서비스할 수 있는 일정 거리를 의미한다. 이 때는 도보 기준 15~20분 거리인 1.5km에 해당한다.
- `TARGET_COVER = 0.90` : 목표 가중 커버율, 즉 주민들이나 지역에서 무인택배함 서비스를 이용할 비율을 의미하며,  $\alpha=0.90$ 인 경우에는, 90% 커버할 수 있도록 설정한 것을 의미한다.
- `POP_W=0.7`, `TRANSIT_W=0.3` : 수요 가중치이며, 인구밀도(POP\_W)와 교통량(TRANSIT\_W)을 각각 7 : 3의 비율로 설정하였다.

## 결과

위 최적화 수리 모델을 실행한 결과, 대전 대덕구에 필요한 무인 택배함의 수는 최소 4개이다.

만약 대덕구에 신규 4개의 무인택배함을 설치한다면, 1.5 km 이내에서 가중 수요의 90% 이상이 무인 택배함 서비스를 이용할 수 있다는 사실을 발견할 수 있었다.

```

Optimal solution found (tolerance 1.00e-04)
Best objective 4.000000000000e+00, best bound 4.000000000000e+00, gap 0.0000%
[RESULT] 최소 설치 개수 p = 4 (가중 커버리지 = 90.8%, R=1500m)

```

1단계 수리 모델을 실행한 결과, 대덕구에 필요한 무인택배함 수를 알 수 있다.

이와 같이 커버 임계거리(T), 목표 가중 커버율, 수요 가중치 비율(POP\_W(인구밀도 가중치), TRANSIT\_W(교통량 가중치))을 수정해가며 대덕구에 신규 설치할 무인택배함 개수(p)를 구할 수 있다.

\* 실제로 서구에도 커버 임계거리(T)를 1.5km로 설정한 후 같은 수리 모델을 이용하면 다음과 같은 결과를 얻을 수 있다.

```

# -*- coding: utf-8 -*-
import geopandas as gpd
import pandas as pd, numpy as np
from shapely.ops import unary_union
import gurobipy as gp
from gurobipy import GRB

```

```

# ----- 입력 -----
R_COVER_M      = 1500      # 커버 반경 (m)
TARGET_COVER    = 0.90      # 목표 커버리지(가중치 기준 90%)

POP_W, TRANSIT_W = 0.7, 0.3  # w_i 가중치

# ----- 1) 데이터 준비: 대덕구 격자/가중치 w_i -----
gdf = gpd.read_file(daejeok_grid_shp)
gdf = gdf[gdf.geometry.notnull() & gdf.geometry.is_valid].copy()
if (not gdf.crs) or (not gdf.crs.is_projected):
    gdf = gdf.to_crs(5179)

pop_col = "val"
gdf[pop_col] = pd.to_numeric(gdf[pop_col], errors="coerce").fillna(0)

df_pts = pd.read_csv(transit_csv)
df_pts = df_pts[df_pts["시군구명"].astype(str).str.contains("서구", na=False)].copy()
gdf_pts = gpd.GeoDataFrame(df_pts,
                           geometry=gpd.points_from_xy(df_pts["경도"], df_pts["위도"]),
                           crs=4326).to_crs(5179)

joined = gpd.sjoin(gdf_pts, gdf[["geometry"]], how="left", predicate="intersects")
joined["월 평균 승하차 인원"] = pd.to_numeric(joined["월 평균 승하차 인원"], errors="coerce").fillna(0)
bus_mean = joined[joined["시설 종류"]=="버스"].groupby("index_right")["월 평균 승하차 인원"].mean()
sub_mean = joined[joined["시설 종류"]=="지하철"].groupby("index_right")["월 평균 승하차 인원"].mean()

gdf["bus_mean"] = gdf.index.map(bus_mean).fillna(0)
gdf["sub_mean"] = gdf.index.map(sub_mean).fillna(0)
gdf["transit"] = gdf["bus_mean"] + gdf["sub_mean"]

# Min-Max → w_i
def minmax(s):
    s = s.astype(float)
    lo, hi = s.min(), s.max()
    return pd.Series(0.0, s.index) if hi-lo < 1e-12 else (s-lo)/(hi-lo)

gdf["pop_n"] = minmax(gdf[pop_col])
gdf["tr_n"] = minmax(gdf["transit"])
gdf["w"] = POP_W*gdf["pop_n"] + TRANSIT_W*gdf["tr_n"]

# 수요점은 격자 중심
gdf_cent = gdf.copy()
gdf_cent["geometry"] = gdf_cent.geometry.centroid
l = list(range(len(gdf_cent)))

```

```

# ----- 2) 후보지 준비 -----
df_c = pd.read_csv(cands_csv, encoding="utf-8-sig")
gdf_c = gpd.GeoDataFrame(df_c,
    geometry=gpd.points_from_xy(df_c["경도"], df_c["위도"]),
    crs=4326).to_crs(5179)
J = list(range(len(gdf_c)))

# ----- 3) 커버 행렬 a_ij (within R_COVER_M) -----
ci = np.array([(p.x, p.y) for p in gdf_cent.geometry])
cj = np.array([(p.x, p.y) for p in gdf_c.geometry])
dmat = np.sqrt(((ci[:,None,:] - cj[None,:,:])**2).sum(axis=2)) # ||x||J
A = (dmat <= R_COVER_M).astype(int) # a_ij

# ----- 4) 최소 p로 목표 커버리지 달성 (maximize covered, then binary search p or minimize Sum x) -----
# 여기서는 '최소 p'를 직접 최적화: Minimize Sum x, with weighted coverage constraint
Wtot = float(gdf_cent["w"].sum())

m = gp.Model("min_p_for_target_coverage")
x = m.addVars(J, vtype=GRB.BINARY, name="x") # 후보 설치 여부
z = m.addVars(I, vtype=GRB.BINARY, name="z") # 수요 i가 커버되면 1

# 커버 논리: sum_j a_ij x_j >= z_i
for i in I:
    m.addConstr(gp.quicksum(A[i,j]*x[j] for j in J) >= z[i])

# 목표 커버리지 (가중치 기준)
m.addConstr(gp.quicksum(gdf_cent.loc[i,"w"] * z[i] for i in I) >= TARGET_COVER * Wtot)

# 목표: 설치 개수 최소화
m.setObjective(gp.quicksum(x[j] for j in J), GRB.MINIMIZE)
m.Params.TimeLimit = 300
m.optimize()

open_sites = [j for j in J if x[j].X > 0.5]
p_min = len(open_sites)
covered_weight = sum(gdf_cent.loc[i,"w"] for i in I if z[i].X > 0.5)
print(f"[RESULT] 최소 설치 개수 p = {p_min} (가중 커버리지 = {covered_weight/Wtot:.1%}, R={R_COVER_M}m)")eo

```

## 결과

결과적으로, 서구에서 필요한 무인택배함의 최소 설치 개수는 6개이며 가장 커버리지 비율은 94.8%임을 알 수 있었다. 이는 서구에 이미 설치된 무인택배함 수와 일치하였고, 반경 1.5 km를 커버 임계거리로 설정한 값이 현실적인 서비스 범위를 적절히 반영하는 것으로 판단하는 계기가 되었다.

## 수리모형 - 2단계: p-median 기반 최적 입지선정

서구에 이미 존재하는 무인 택배함의 수를 고정하고, 대덕구에서는 1단계에서 결정된 필요한 최소한의 설치 수 ( $p=4$ ) 를 입력하여, 신규 설치 후보지를 통합적으로 고려할 때, 수요지점  $i$ 와 무인 택배함의 위치 간의 거리를 최소화하는 시설 배치를 결정하였다.

$p$ -median을 기반으로 한 최적 입지 선정에 대한 수리 모형의 변수와 목적함수, 제약식은 다음과 같다.

## 정의

Sets and parameters:

- $I$ : set of demand points
- $J$ : set of candidate facility points
- $p$ : Number of facilities
- $w_i$ : Weight of demand point  $i$
- $d_{ij}$ : distance between demand point  $i$  and candidate facility point  $j$

Decision variables:

- $x_j$ : 1 if a facility is located at candidate facility, 0 otherwise, where  $j \in J$
- $y_{ij}$ : 1 if demand point  $i \in I$  is assigned to facility  $j \in J$

## 수리모형

$$\min \sum_{i \in I} \sum_{j \in J} w_i d_{ij} y_{ij} \quad (1)$$

$$\text{s.t.} \quad \sum_{j \in J} x_j = p \quad (2)$$

$$\sum_{j \in J} y_{ij} = 1 \quad \forall i \in I \quad (3)$$

$$y_{ij} \leq x_j \quad \forall i \in I, \forall j \in J \quad (4)$$

$$x_j \in \{0, 1\}, y_{ij} \in \{0, 1\} \quad (5)$$

- (1) 목적함수는 전체 수요지점과 후보지 간의 가중 거리합을 최소화하는 것이다.
- (2) 설치할 후보지의 개수를  $p$ 개로 제한한다.
- (3) 각 수요지점은 반드시 하나의 후보지에만 할당된다.
- (4) 선택되지 않은 후보지에는 수요를 배정할 수 없다.
- (5)  $x, y$ 는 이진변수이다.

서구 및 대덕구 지역 내 무인택배함 입지를 효율적으로 재배치하기 위해, 인구와 교통(버스·지하철) 가중치를 각각 7:3. 5:5. 3:7 세 가지 시나리오를 설정하였다.

2단계 수리 모형을 코드로 구현한 결과는 다음과 같았다.

## 코드 구현 - 2단계 [서구]

```

import pandas as pd
import numpy as np
import geopandas as gpd
import matplotlib.pyplot as plt
import contextily as cx
import gurobipy as gp
from gurobipy import GRB
import warnings
from matplotlib import font_manager, rc

warnings.filterwarnings("ignore", category=UserWarning)

input_shp = r"인구밀도_서구.shp"
csv_path = r"preprocessing ver7.csv"
current_lockers = r"무인택배함.csv"
new_lockers_path = r"finaldestination.csv"

# ===== 한글 폰트 설정 =====
rc('font', family='AppleGothic') # mac 버전/
plt.rcParams['axes.unicode_minus'] = False

```

### 1. [서구] 인구 0.7 : 교통 (버스+지하철) 0.3

```

# 가중치: 인구 0.7, 교통(버스+지하철) 0.3
POP_W = 0.7
TRANSIT_W = 0.3

# 최적화 파라미터
TIME_LIMIT = 300 # 초
MIP_GAP = 0.01 # 1%

# =====
# 2) 데이터 로드 & 수요 w_i 구성
# =====
print("[INFO] Load Seogu grid ...")
gdf = gpd.read_file(input_shp)
gdf = gdf[gdf.geometry.notnull() & gdf.geometry.is_valid].copy()
if (not gdf.crs) or (not gdf.crs.is_projected):
    gdf = gdf.to_crs(5179) # GRS80 / UTM-K (m 단위 근사)

# 인구 컬럼: 'val' 가정
pop_col = "val"
if pop_col not in gdf.columns:
    raise ValueError(f"'val' 컬럼이 필요합니다. 현재 컬럼: {list(gdf.columns)}")
gdf[pop_col] = pd.to_numeric(gdf[pop_col], errors="coerce").fillna(0)

```



```

print("[INFO] Load transit points (bus/subway) ...")
df_pts = pd.read_csv(csv_path)
df_pts = df_pts[df_pts["시군구명"].str.contains("서구", na=False)].copy()
gdf_pts = gpd.GeoDataFrame(
    df_pts, geometry=gpd.points_from_xy(df_pts["경도"], df_pts["위도"]), crs=4326
).to_crs(5179)

# 격자별 버스/지하철 평균 이용량 집계
joined = gpd.sjoin(gdf_pts, gdf, how="left", predicate="intersects")
joined["월 평균 승하차 인원"] = pd.to_numeric(joined["월 평균 승하차 인원"], errors="coerce").fillna(0)

bus_mean = joined[joined["시설 종류"]=="버스"].groupby("index_right")["월 평균 승하차 인원"].mean()
sub_mean = joined[joined["시설 종류"]=="지하철"].groupby("index_right")["월 평균 승하차 인원"].mean()

gdf["bus_mean"] = gdf.index.map(bus_mean).fillna(0)
gdf["sub_mean"] = gdf.index.map(sub_mean).fillna(0)

# Min-Max 정규화
def minmax(s: pd.Series) → pd.Series:
    s = s.astype(float)
    vmin, vmax = s.min(), s.max()
    if vmax - vmin < 1e-12:
        return pd.Series(0.0, index=s.index)
    return (s - vmin) / (vmax - vmin)

# 정규화 및 w_i 계산: w = 0.7*pop_n + 0.3*transit_n (transit = bus/sub 평균)
gdf["pop_n"] = minmax(gdf["pop_col"])
gdf["bus_n"] = minmax(gdf["bus_mean"])
gdf["sub_n"] = minmax(gdf["sub_mean"])
gdf["transit_n"] = 0.5*(gdf["bus_n"] + gdf["sub_n"])
gdf["w"] = POP_W*gdf["pop_n"] + TRANSIT_W*gdf["transit_n"]

# 수요점 = 격자 중심
gdf_cent = gdf.copy()
gdf_cent["geometry"] = gdf_cent.geometry.centroid # 5179 기준
l = list(range(len(gdf_cent)))

# =====
# 3) 후보 집합 J, p 설정 (현재 개수 유지, 새 후보만 선택)
# =====
print("[INFO] Load current & candidate lockers ...")
df_cur = pd.read_csv(current_lockers, encoding="utf-8-sig")
gdf_cur = gpd.GeoDataFrame(
    df_cur, geometry=gpd.points_from_xy(df_cur["경도"], df_cur["위도"]), crs=4326
).to_crs(5179)

```

```

df_new = pd.read_csv(new_lockers_path, encoding="utf-8-sig")
gdf_cand = gpd.GeoDataFrame(
    df_new, geometry=gpd.points_from_xy(df_new["경도"], df_new["위도"]), crs=4326
).to_crs(5179)

# 설치 개수 p = 현재 지점 수
p = len(gdf_cur)
J = list(range(len(gdf_cand)))
print(f"[INFO] |I|={len(I)}, |J|={len(J)}, p={p}")

# =====
# 4) 거리행렬 (5179: m 근사)
# =====
ci = np.array([(pt.x, pt.y) for pt in gdf_cent.geometry])
cj = np.array([(pt.x, pt.y) for pt in gdf_cand.geometry])
dmat = np.sqrt(((ci[:,None,:] - cj[None,:,:])**2).sum(axis=2)) # |I| x |J|

# =====
# 5) p-median 최적화
# =====
print("[INFO] Solve p-median (w = 0.7*pop + 0.3*transit) ...")
w = gdf_cent["w"].to_dict()

m = gp.Model("p_median_relocation")
x = m.addVars(J, vtype=GRB.BINARY, name="x") # 후보 설치 여부
y = m.addVars([(i,j) for i in I for j in J], vtype=GRB.BINARY, name="y") # i→j 할당

# 설치 개수 고정
m.addConstr(gp.quicksum(x[j] for j in J) == p)
# 각 수요 i는 정확히 1곳에 할당
for i in I:
    m.addConstr(gp.quicksum(y[i,j] for j in J) == 1)
# 설치된 곳에만 할당
for i in I:
    for j in J:
        m.addConstr(y[i,j] <= x[j])

# 목적:  $\sum_i \sum_j w_i * d_{ij} * y_{ij}$  최소화
m.setObjective(gp.quicksum(w[i]*dmat[i,j]*y[i,j] for i in I for j in J), GRB.MINIMIZE)

m.Params.OutputFlag = 1
m.Params.TimeLimit = TIME_LIMIT
m.Params.MIPGap = MIP_GAP
m.optimize()

open_sites = [j for j in J if x[j].X > 0.5]
assign = {i: max(J, key=lambda j: y[i,j].X) for i in I}

```

```

print(f"[RESULT] Open sites (count={len(open_sites)}): {open_sites}")

# =====
# 6) 목적함수 비교: 현행 vs 최적
# =====
# 현행 설치지 집합(현재 무인택배함)과 최적 설치지(새 후보 중 선택)에 대해
#  $\sum_i w_i * \min_j d(i, j)$  값을 각각 계산
print("\n=== 목적함수 비교 (p-median;  $\sum_i w_i * d(i, \text{시설})$ ) ===")
ci_xy = np.array([(pt.x, pt.y) for pt in gdf_cent.geometry]) # |I| x 2
cj_cur_xy = np.array([(pt.x, pt.y) for pt in gdf_cur.geometry]) # |J_cur| x 2
cj_opt_xy = np.array([(pt.x, pt.y) for pt in gdf_cand.iloc[open_sites].geometry]) # |p| x 2

dmat_cur = np.sqrt(((ci_xy[:,None,:] - cj_cur_xy[None,:,:])**2).sum(axis=2)) if len(cj_cur_xy)>0
else np.zeros((len(ci_xy),1))
dmat_opt = np.sqrt(((ci_xy[:,None,:] - cj_opt_xy[None,:,:])**2).sum(axis=2)) if len(cj_opt_xy)>0
else np.zeros((len(ci_xy),1))

min_d_cur = dmat_cur.min(axis=1) if dmat_cur.shape[1] > 0 else np.zeros(len(ci_xy))
min_d_opt = dmat_opt.min(axis=1) if dmat_opt.shape[1] > 0 else np.zeros(len(ci_xy))

w_vec = gdf_cent["w"].to_numpy()
obj_cur = float(np.dot(w_vec, min_d_cur))
obj_opt = float(np.dot(w_vec, min_d_opt))
improve_abs = obj_cur - obj_opt
improve_pct = (improve_abs / obj_cur * 100.0) if obj_cur > 1e-12 else 0.0

print(f"현행 설치(현재 무인택배함) : {obj_cur:,.2f}")
print(f"최적 해(새 후보 중 선택) : {obj_opt:,.2f}")
print(f"차이(현행-최적) : {improve_abs:,.2f}")
print(f"개선율(%) : {improve_pct:,.2f}%")

# =====
# 7) 시각화 & 저장 (서구로 자동 확대)
# =====
print("[INFO] Plot ...")
gdf_3857 = gdf.to_crs(3857)
cand_3857 = gdf_cand.to_crs(3857)
cur_3857 = gdf_cur.to_crs(3857)

fig, ax = plt.subplots(figsize=(13, 13))

# 배경: w 코로플레스 (인구0.7+교통0.3)
gdf_3857.plot(ax=ax, column="w", cmap="YlOrRd", legend=True,
              edgecolor="gray", linewidth=0.2, alpha=0.85)

# 전체 후보(연회색 점)
cand_3857.plot(ax=ax, color="#9e9e9e", markersize=18, alpha=0.55, label="후보 전체", zorder=
6)

```

```

# 현재 지점(빨간★)
cur_3857.plot(ax=ax, marker="*", color="red", edgecolor="black",
              markersize=160, linewidth=0.6, alpha=0.9, label="현재 지점", zorder=8)

# 선택 지점(금색★)
chosen = cand_3857.iloc[open_sites]
chosen.plot(ax=ax, marker="*", color="gold", edgecolor="black",
            markersize=200, linewidth=0.7, alpha=0.95, label="선택 지점 (p-median)", zorder=9)

# ---- 서구 격자 바운딩박스만으로 축 설정(과도한 확대/축소 방지) ----
xmin, ymin, xmax, ymax = gdf_3857.total_bounds
pad_x = (xmax - xmin) * 0.05 # 5% 여백
pad_y = (ymax - ymin) * 0.05
ax.set_xlim(xmin - pad_x, xmax + pad_x)
ax.set_ylim(ymin - pad_y, ymax + pad_y)

# 베이스맵
cx.add_basemap(ax, crs=gdf_3857.crs, source=cx.providers.CartoDB.Positron)

ax.set_title(
    f"서구 무인택배함 p-median (인구 0.7 + 교통 0.3)\n"
    f"현행: {obj_cur:,.0f} → 최적: {obj_opt:,.0f} (Δ {improve_abs:,.0f}, {improve_pct:,.2f}%)",
    fontsize=14, fontweight="bold"
)
ax.legend(loc="lower right")
ax.set_aspect("equal")
plt.tight_layout()
plt.savefig("seogu_pmedian_pop0p7_transit0p3.png", dpi=300, bbox_inches="tight")
plt.show()

```

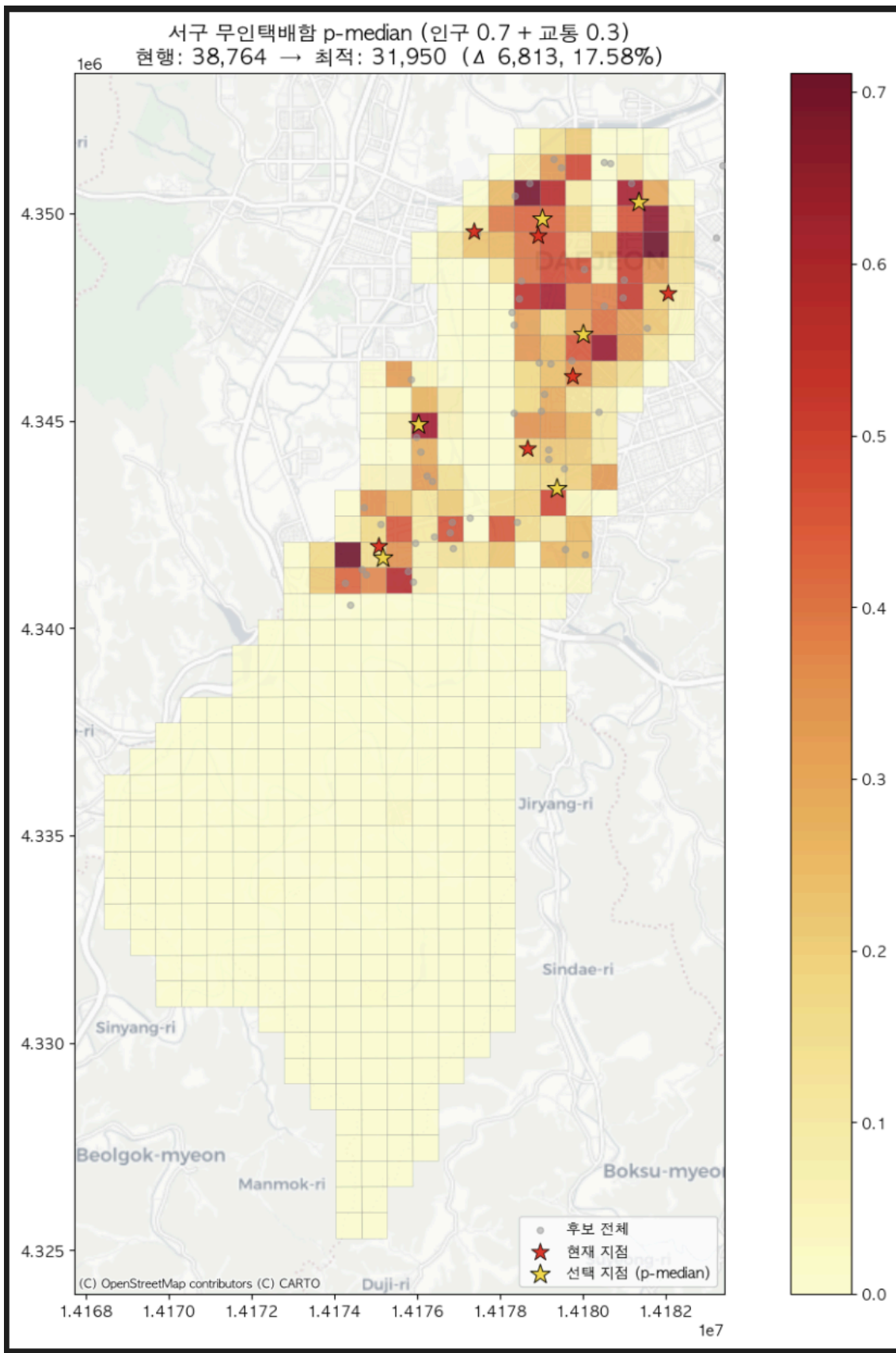
## 입력 및 기본 설정

- `input_shp`, `csv_path`, `current_lockers`, `new_lockers_path` :  
서구의 인구 밀도 격자(250m x 250m) 데이터, 버스와 지하철의 위치 데이터, 무인 택배함의 현행 설치 지점, 신규 무인택배함 설치 후보지에 대한 파일 경로를 의미한다.
- `POP_W = 0.7`, `TRANSIT_W = 0.3` : 인구 밀도 수요 가중치(POP\_W)와 교통량의 가중치(TRANSIT\_W)를 의미하며, 각각 7 : 3의 비율을 가진다.
- `p = len(gdf_cur)`  
: 서구의 경우에는 1단계에 도출된 결과인 현행 무인택배함 설치 개수(6개)를 동일하게 유지한다.  
대덕구의 경우에는 1단계에서 도출된 최소한의 무인택배함 설치 개수( $p=4$ )를 입력 값으로 사용한다.

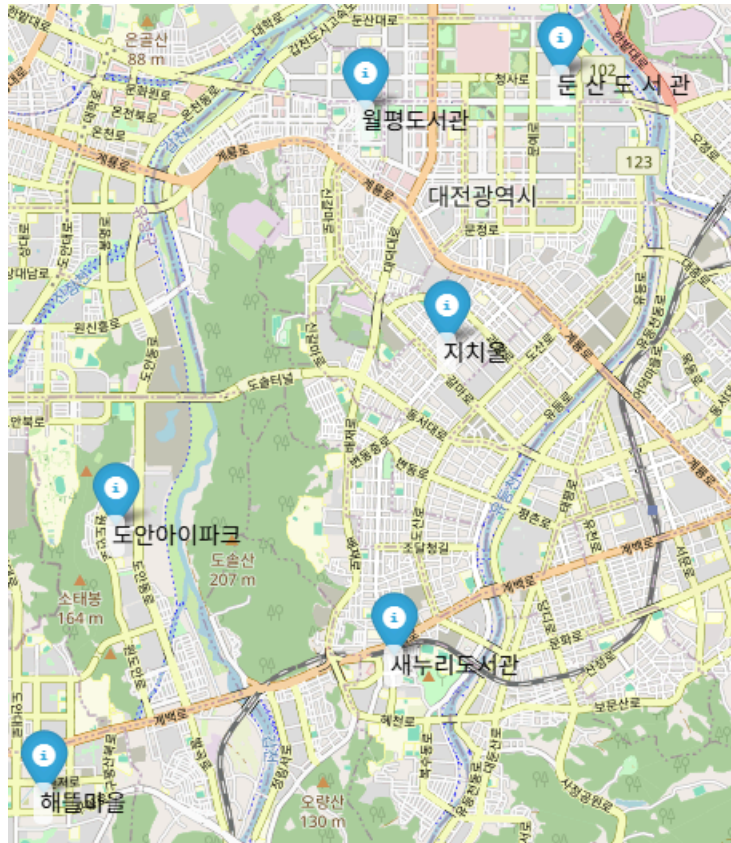
## 결과값

- 인구 밀도 수요 가중치와 교통량 가중치를 각각 0.7과 0.3으로 설정한 결과는 다음과 같았다.

- 새 후보지 중 6개의 후보지를 선택하였을 때의 목적함수의 값은 31,950.31과 같았다.  
기존의 목적함수의 값과 6개 후보지를 선택하였을 때의 목적함수 값의 차이는 6,813.26였다.
- 즉, 17.6%의 효율 개선이 이뤄졌으며, 현행 대비 총 이동거리가 6,813만큼 감소하였음을 알 수 있었다.



서구에 인구밀도 : 교통량 비율을 7:3으로 적용한 결과



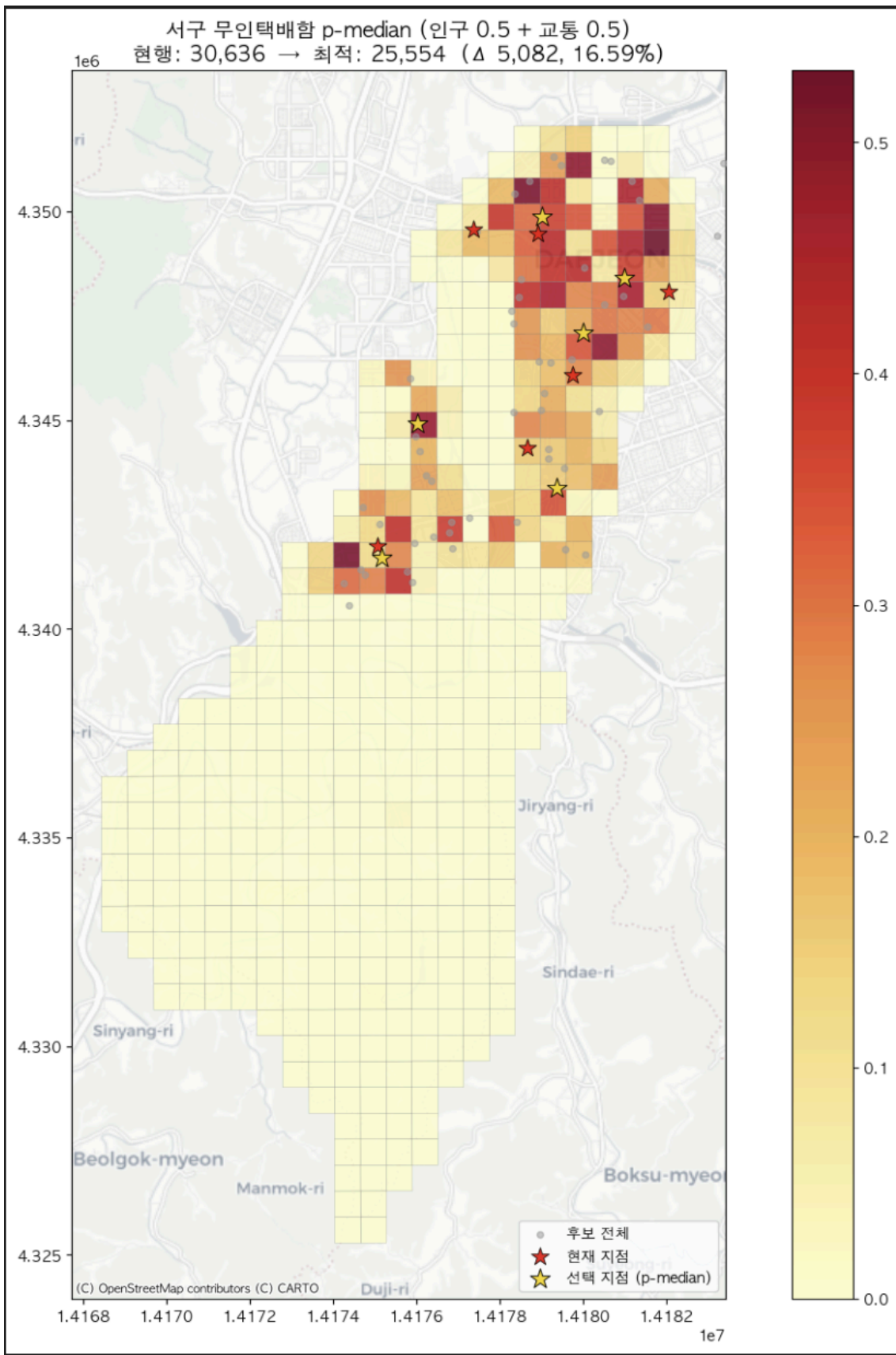
실제 대전 서구 지도(인구밀도 : 교통량 = 7:3)에 대입하여 시각화한 사진

## 2. [서구] 인구 0.5 : 교통 (버스+지하철) 0.5

# 가중치: 인구 0.5, 교통(버스+지하철) 0.5  
 POP\_W = 0.5  
 TRANSIT\_W = 0.5

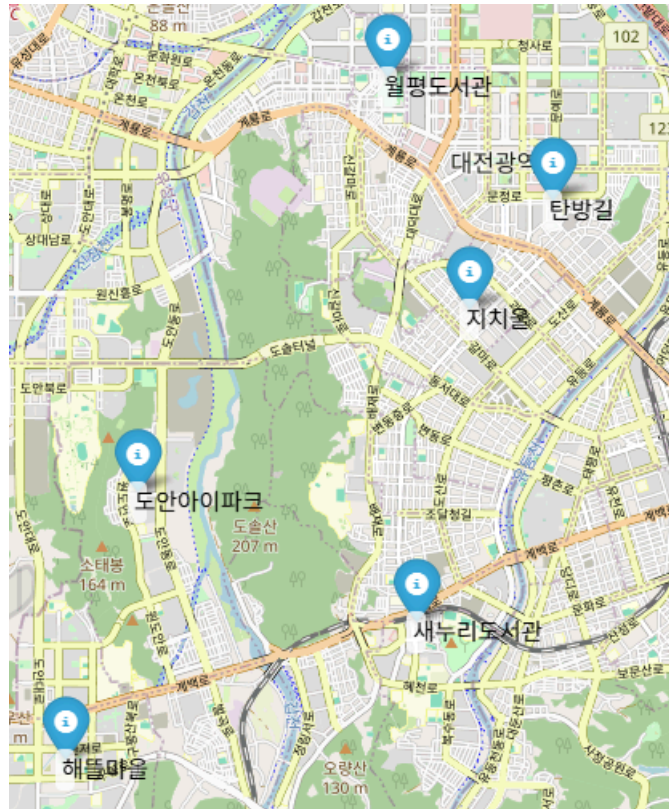
### 결과값

- 인구 밀도 수요 가중치와 교통량 가중치를 각각 0.5과 0.5으로 설정한 결과는 다음과 같았다.
- 새 후보지 중 6개의 후보지를 선택하였을 때의 목적함수의 값은 25,554.10과 같았다.  
 기존의 목적함수의 값과 6개 후보지를 선택하였을 때의 목적함수 값의 차이는 5,082.36였다.
- 즉, 16.59%의 효율 개선이 이뤄졌으며, 현행 대비 총 이동거리가 5,082만큼 감소하였음을 알 수 있었다.



서구에 인구밀도 : 교통량 비율을 5:5으로 적용한 결과





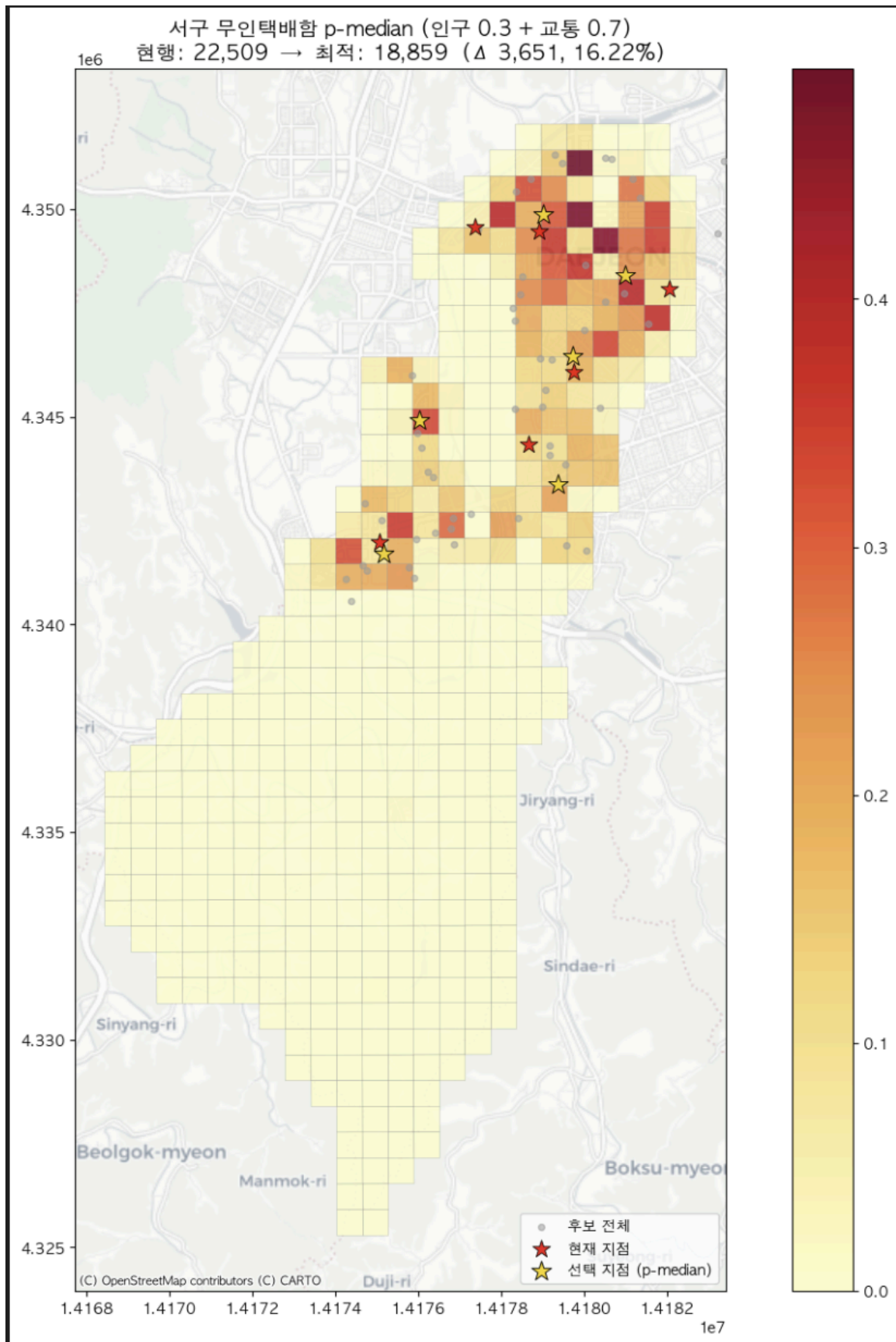
실제 대전 서구 지도(인구밀도: 교통량 = 5:5)에 대입하여 시각화한 사진

### 3. [서구] 인구 0.3 : 교통 (버스+지하철) 0.7

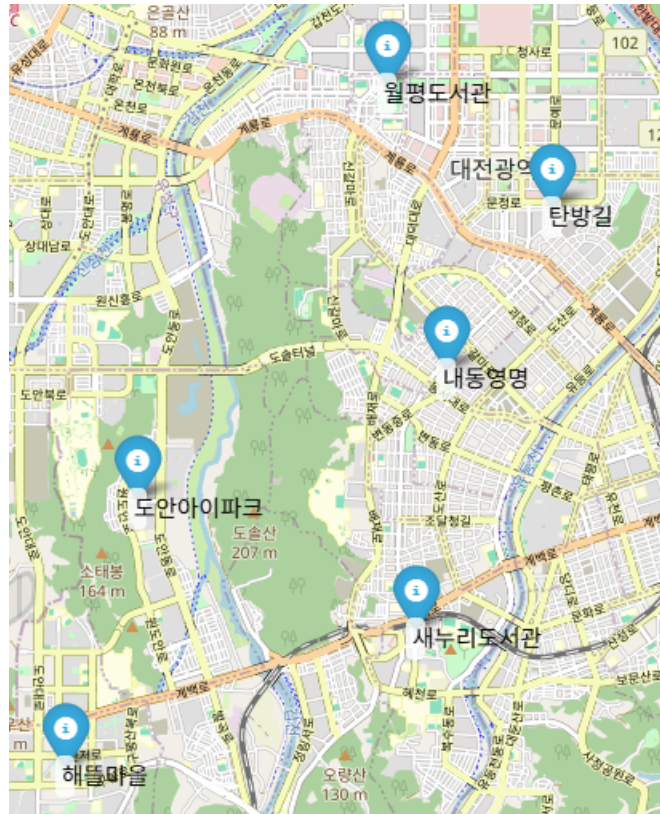
# 가중치: 인구 0.5, 교통(버스+지하철) 0.5  
 POP\_W = 0.3  
 TRANSIT\_W = 0.7

#### 결과값

- 인구 밀도 수요 가중치와 교통량 가중치를 각각 0.3과 0.7으로 설정한 결과는 다음과 같았다.
- 새 후보지 중 6개의 후보지를 선택하였을 때의 목적함수의 값은 18,858.69와 같았다.  
 기존의 목적함수의 값과 6개 후보지를 선택하였을 때의 목적함수 값의 차이는 3,650.64였다.
- 즉, 16.22%의 효율 개선이 이뤄졌으며, 현행 대비 총 이동거리가 3,650 만㎞ 감소하였음을 알 수 있었다.



서구에 인구밀도 : 교통량 비율을 3:7으로 적용한 결과



실제 대전 서구 지도(인구밀도: 교통량 = 3:7)에 대입하여 시각화한 사진

## 1단계 분석 결과

- 수립한 수리 모델을 바탕으로 분석한 결과, 대전 서구에 교통량보다 인구 밀도에 가중치를 더 부여하였을 때, 거주 밀집 지역에 무인 택배함이 다수 배치되었다.
- 전체 이동거리가 17.6% 감소하며, 생활권 중심의 균형적 무인택배함 커버 구조를 확보하였다. 이는 인구 기반 서비스 접근성을 최적화하는 데 효과적인 결과로 해석된다.

## 코드 구현 - 2단계 [대덕구]

```
import pandas as pd
import numpy as np
import geopandas as gpd
import matplotlib.pyplot as plt
import contextily as cx
import gurobipy as gp
from gurobipy import GRB
import warnings
from matplotlib import font_manager, rc

warnings.filterwarnings("ignore", category=UserWarning)

input_shp = r"../인구밀도_대덕구.shp"
csv_path = r"../preprocessing ver7.csv"
```

```
current_lockers = r"..../무인택배함.csv"
new_lockers_path = r"..../finaldestination.csv"

# ===== 한글 폰트 설정 =====
rc('font', family='AppleGothic') # mac 버전/
plt.rcParams['axes.unicode_minus'] = False
```

### 1. [대덕구] 인구 0.7 : 교통 (버스+지하철) 0.3

```
#가중치: 인구 0.7, 교통(버스+지하철) 0.3
POP_W = 0.7
TRANSIT_W = 0.3

# 최적화 파라미터
TIME_LIMIT = 300 # 초
MIP_GAP = 0.01 # 1%

# =====
# 2) 데이터 로드 & 수요 w_i 구성
# =====
print("[INFO] Load Seogu grid ...")
gdf = gpd.read_file(input_shp)
gdf = gdf[gdf.geometry.notnull() & gdf.geometry.is_valid].copy()
if (not gdf.crs) or (not gdf.crs.is_projected):
    gdf = gdf.to_crs(5179) # GRS80 / UTM-K (m 단위 근사)

# 인구 컬럼: 'val' 가정
pop_col = "val"
if pop_col not in gdf.columns:
    raise ValueError(f"'val' 컬럼이 필요합니다. 현재 컬럼: {list(gdf.columns)}")
gdf[pop_col] = pd.to_numeric(gdf[pop_col], errors="coerce").fillna(0)

print("[INFO] Load transit points (bus/subway) ...")
df_pts = pd.read_csv(csv_path)
df_pts = df_pts[df_pts["시군구명"].str.contains("대덕구", na=False)].copy()
gdf_pts = gpd.GeoDataFrame(
    df_pts, geometry=gpd.points_from_xy(df_pts["경도"], df_pts["위도"]), crs=4326
).to_crs(5179)

# 격자별 버스/지하철 평균 이용량 집계
joined = gpd.sjoin(gdf_pts, gdf, how="left", predicate="intersects")
joined["월 평균 승하차 인원"] = pd.to_numeric(joined["월 평균 승하차 인원"], errors="coerce").fillna(0)

bus_mean = joined[joined["시설 종류"]=="버스"].groupby("index_right")["월 평균 승하차 인원"].mean()
sub_mean = joined[joined["시설 종류"]=="지하철"].groupby("index_right")["월 평균 승하차 인원"].
```

```

mean()

gdf["bus_mean"] = gdf.index.map(bus_mean).fillna(0)
gdf["sub_mean"] = gdf.index.map(sub_mean).fillna(0)

# Min-Max 정규화
def minmax(s: pd.Series) → pd.Series:
    s = s.astype(float)
    vmin, vmax = s.min(), s.max()
    if vmax - vmin < 1e-12:
        return pd.Series(0.0, index=s.index)
    return (s - vmin) / (vmax - vmin)

# 정규화 및 w_i 계산: w = 0.7*pop_n + 0.3*transit_n (transit = bus/sub 평균)
gdf["pop_n"] = minmax(gdf[pop_col])
gdf["bus_n"] = minmax(gdf["bus_mean"])
gdf["sub_n"] = minmax(gdf["sub_mean"])
gdf["transit_n"] = 0.5*(gdf["bus_n"] + gdf["sub_n"])
gdf["w"] = POP_W*gdf["pop_n"] + TRANSIT_W*gdf["transit_n"]

# 수요점 = 격자 중심
gdf_cent = gdf.copy()
gdf_cent["geometry"] = gdf_cent.geometry.centroid # 5179 기준
l = list(range(len(gdf_cent)))

# =====
# 3) 후보 집합 J, p 설정 (현재 개수 유지, 새 후보만 선택)
# =====
print("[INFO] Load current & candidate lockers ...")
df_cur = pd.read_csv(current_lockers, encoding="utf-8-sig")
gdf_cur = gpd.GeoDataFrame(
    df_cur, geometry=gpd.points_from_xy(df_cur["경도"], df_cur["위도"]), crs=4326
).to_crs(5179)

df_new = pd.read_csv(new_lockers_path, encoding="utf-8-sig")
gdf_cand = gpd.GeoDataFrame(
    df_new, geometry=gpd.points_from_xy(df_new["경도"], df_new["위도"]), crs=4326
).to_crs(5179)

# 설치 개수 p = 현재 지점 수
p = 4
J = list(range(len(gdf_cand)))
print(f"[INFO] |I|={len(l)}, |J|={len(J)}, p={p}")

# =====
# 4) 거리행렬 (5179: m 근사)
# =====
ci = np.array([(pt.x, pt.y) for pt in gdf_cent.geometry])

```

```

cj = np.array([(pt.x, pt.y) for pt in gdf_cand.geometry])
dmat = np.sqrt(((ci[:,None,:] - cj[None,:,:])**2).sum(axis=2)) # |I| x |J|

# =====
# 5) p-median 최적화
# =====
print("[INFO] Solve p-median (w = 0.7*pop + 0.3*transit) ...")
w = gdf_cent["w"].to_dict()

m = gp.Model("p_median_relocation")
x = m.addVars(J, vtype=GRB.BINARY, name="x") # 후보 설치 여부
y = m.addVars(((i,j) for i in I for j in J), vtype=GRB.BINARY, name="y") # i→j 할당

# 설치 개수 고정
m.addConstr(gp.quicksum(x[j] for j in J) == p)
# 각 수요 i는 정확히 1곳에 할당
for i in I:
    m.addConstr(gp.quicksum(y[i,j] for j in J) == 1)
# 설치된 곳에만 할당
for i in I:
    for j in J:
        m.addConstr(y[i,j] <= x[j])

# 목적:  $\sum_i \sum_j w_i * d_{ij} * y_{ij}$  최소화
m.setObjective(gp.quicksum(w[i]*dmat[i,j]*y[i,j] for i in I for j in J), GRB.MINIMIZE)

m.Params.OutputFlag = 1
m.Params.TimeLimit = TIME_LIMIT
m.Params.MIPGap = MIP_GAP
m.optimize()

open_sites = [j for j in J if x[j].X > 0.5]
assign = {i: max(J, key=lambda j: y[i,j].X) for i in I}
print(f"[RESULT] Open sites (count={len(open_sites)}): {open_sites}")

# =====
# 6) 목적함수 비교: 현행 vs 최적
# =====
# 현행 설치지 집합(현재 무인택배함)과 최적 설치지(새 후보 중 선택)에 대해
#  $\sum_i w_i * \min_j d(i, j)$  값을 각각 계산
print("\n=== 목적함수 비교 (p-median;  $\sum_i w_i * d(i, \text{시설})$ ) ===")
ci_xy = np.array([(pt.x, pt.y) for pt in gdf_cent.geometry]) # |I| x 2
cj_cur_xy = np.array([(pt.x, pt.y) for pt in gdf_cur.geometry]) # |J_cur| x 2
cj_opt_xy = np.array([(pt.x, pt.y) for pt in gdf_cand.iloc[open_sites].geometry]) # |p| x 2

dmat_cur = np.sqrt(((ci_xy[:,None,:] - cj_cur_xy[None,:,:])**2).sum(axis=2)) if len(cj_cur_xy)>0
else np.zeros((len(ci_xy),1))
dmat_opt = np.sqrt(((ci_xy[:,None,:] - cj_opt_xy[None,:,:])**2).sum(axis=2)) if len(cj_opt_xy)>0

```

```

else np.zeros((len(ci_xy),1))

min_d_cur = dmat_cur.min(axis=1) if dmat_cur.shape[1] > 0 else np.zeros(len(ci_xy))
min_d_opt = dmat_opt.min(axis=1) if dmat_opt.shape[1] > 0 else np.zeros(len(ci_xy))

w_vec = gdf_cent["w"].to_numpy()
obj_cur = float(np.dot(w_vec, min_d_cur))
obj_opt = float(np.dot(w_vec, min_d_opt))
improve_abs = obj_cur - obj_opt
improve_pct = (improve_abs / obj_cur * 100.0) if obj_cur > 1e-12 else 0.0

print(f"현행 설치(현재 무인택배함) : {obj_cur:,.2f}")
print(f"최적 해(새 후보 중 선택) : {obj_opt:,.2f}")
print(f"차이(현행-최적) : {improve_abs:,.2f}")
print(f"개선율(%) : {improve_pct:,.2f}%")

# =====
# 7) 시각화 & 저장 (서구로 자동 확대)
# =====
print("[INFO] Plot ...")
gdf_3857 = gdf.to_crs(3857)
cand_3857 = gdf_cand.to_crs(3857)
cur_3857 = gdf_cur.to_crs(3857)

fig, ax = plt.subplots(figsize=(13, 13))

# 배경: w 코로플레스 (인구0.7+교통0.3)
gdf_3857.plot(ax=ax, column="w", cmap="YlOrRd", legend=True,
              edgecolor="gray", linewidth=0.2, alpha=0.85)

# 전체 후보(연회색 점)
cand_3857.plot(ax=ax, color="#9e9e9e", markersize=18, alpha=0.55, label="후보 전체", zorder=6)

# 현재 지점(빨간★)
cur_3857.plot(ax=ax, marker="*", color="red", edgecolor="black",
              markersize=160, linewidth=0.6, alpha=0.9, label="현재 지점", zorder=8)

# 선택 지점(금색★)
chosen = cand_3857.iloc[open_sites]
chosen.plot(ax=ax, marker="*", color="gold", edgecolor="black",
            markersize=200, linewidth=0.7, alpha=0.95, label="선택 지점 (p-median)", zorder=9)

# ---- 서구 격자 바운딩박스만으로 축 설정(과도한 확대/축소 방지) ----
xmin, ymin, xmax, ymax = gdf_3857.total_bounds
pad_x = (xmax - xmin) * 0.05 # 5% 여백
pad_y = (ymax - ymin) * 0.05
ax.set_xlim(xmin - pad_x, xmax + pad_x)

```



```

ax.set_ylim(ymin - pad_y, ymax + pad_y)

# 베이스맵
cx.add_basemap(ax, crs=gdf_3857.crs, source=cx.providers.CartoDB.Positron)

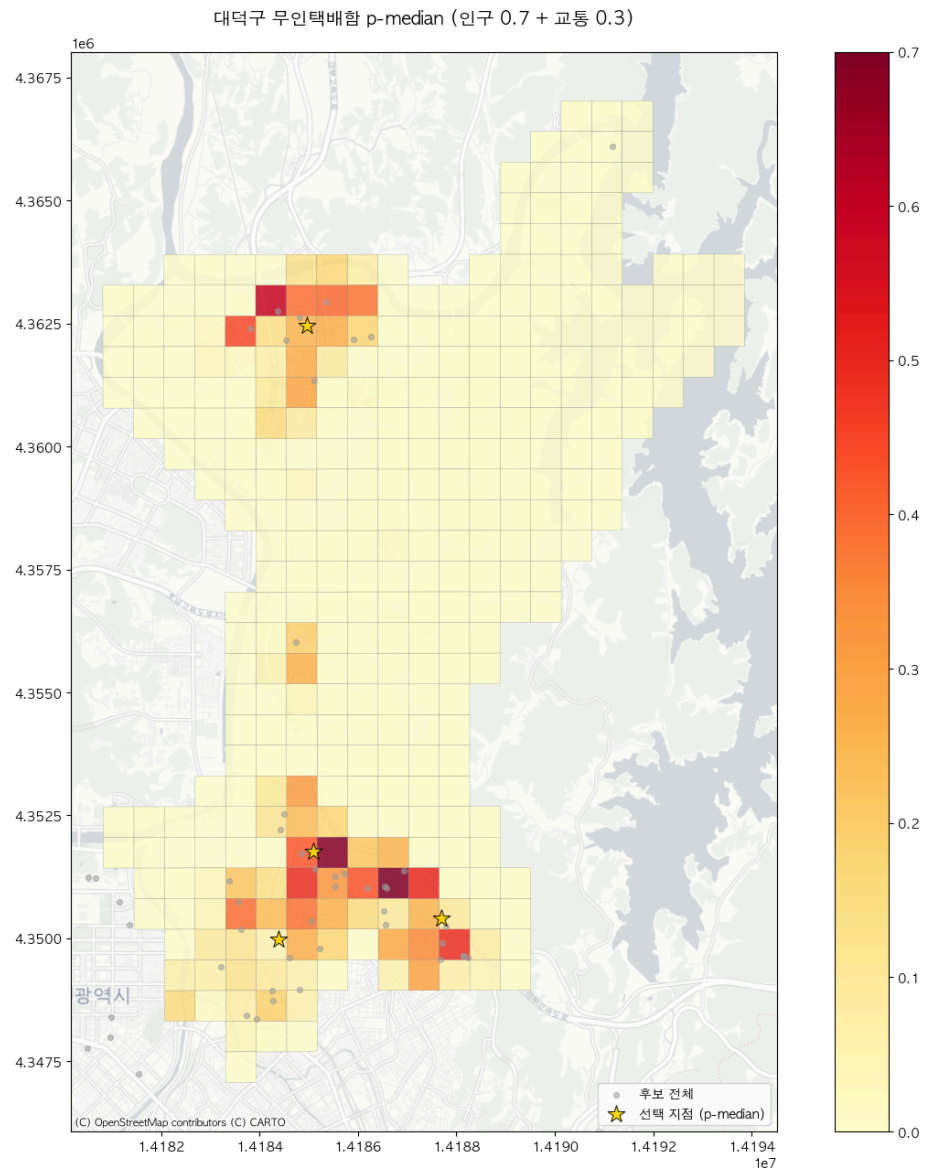
ax.set_title(
    f"대덕구 무인택배함 p-median (인구 0.7 + 교통 0.3)\n"
    f"현행: {obj_cur:,.0f} → 최적: {obj_opt:,.0f} (Δ {improve_abs:,.0f}, {improve_pct:,.2f}%)",
    fontsize=14, fontweight="bold"
)
ax.legend(loc="lower right")
ax.set_aspect("equal")
plt.tight_layout()
plt.savefig("seogu_pmedian_pop0p7_transit0p3.png", dpi=300, bbox_inches="tight")
plt.show()

```

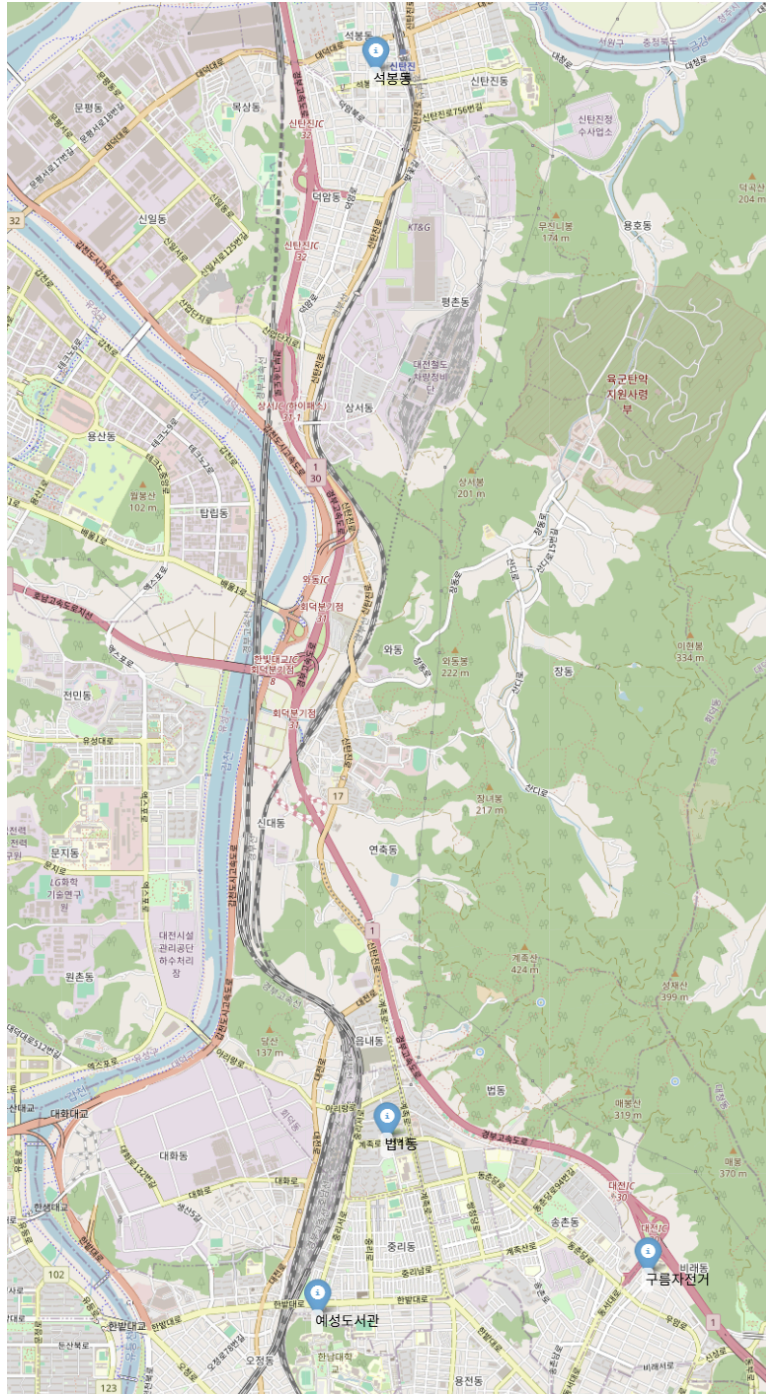
대덕구의 경우 서구와 달리 신규 설치 지역을 탐색하기 위한 것이기 때문에 기존 목적함수 값과 새로운 목적 함수 값을 비교하기보다는 다음과 같이 최적화된 신규 설치 지역의 위치를 찾는 것에 중점을 두었다.

## 결과





대덕구에 인구밀도 : 유동인구 비율을 7:3으로 적용한 결과



실제 대전 대덕구 지도(인구밀도: 교통량 = 7:3)에 대입하여 시각화한 사진

## 2. [대덕구] 인구 0.5 : 교통 (버스+지하철) 0.5

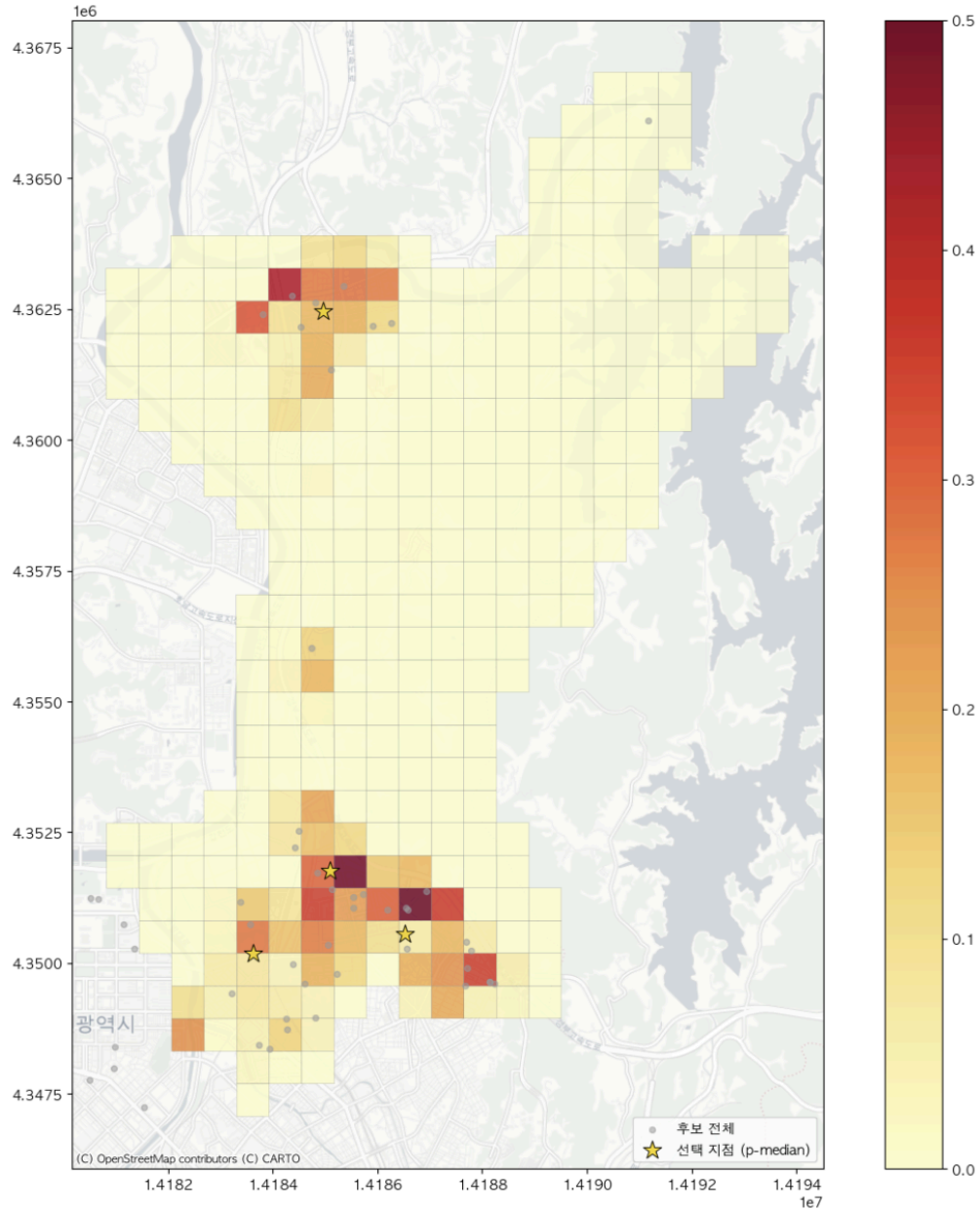
#가중치: 인구 0.5, 교통(버스+지하철) 0.5

POP\_W = 0.5

TRANSIT\_W = 0.5

## 결과

대덕구 무인택배함 p-median (인구 0.5 + 교통 0.5)



대덕구에 인구밀도 : 유동인구 비율을 5:5으로 적용한 결과



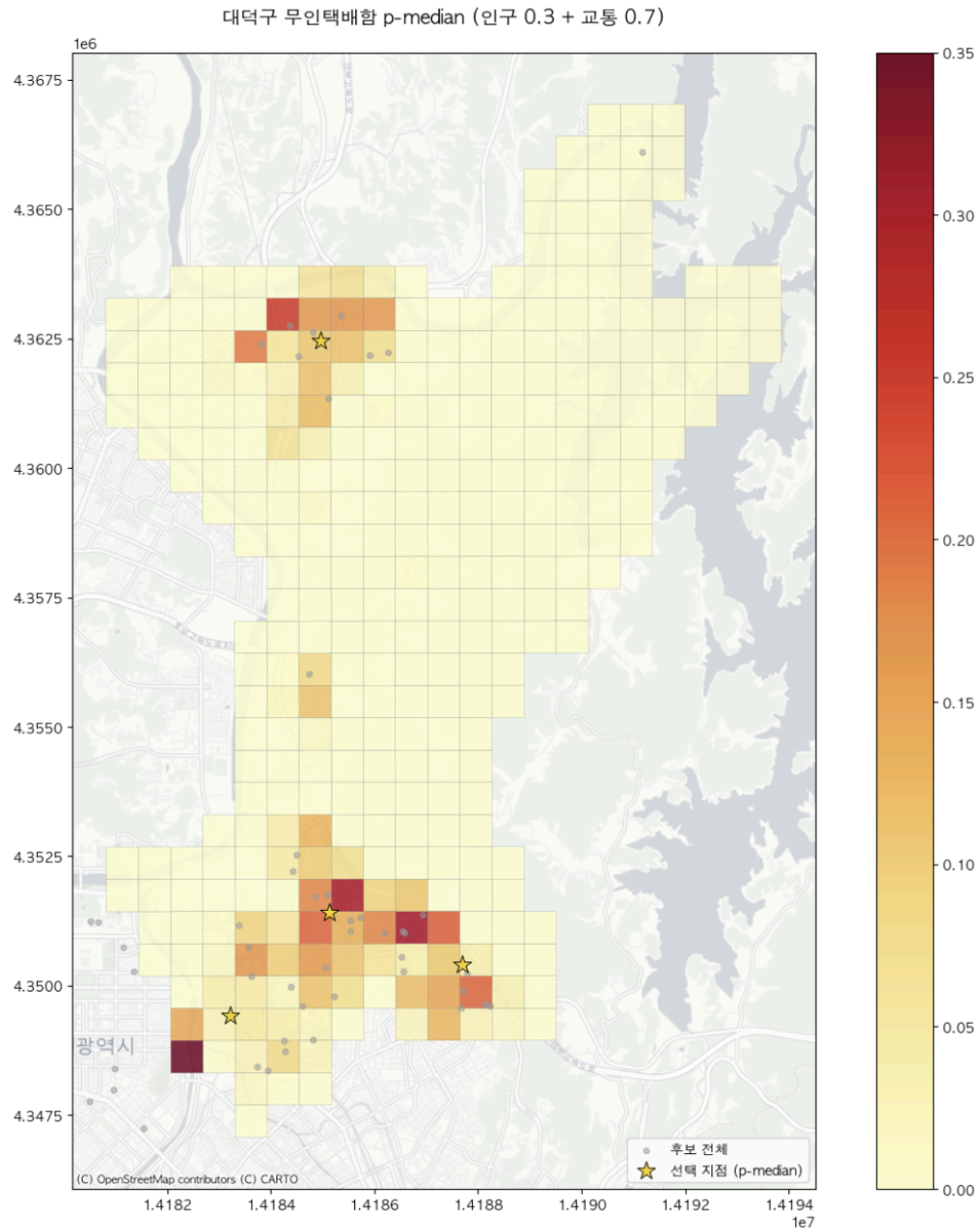


실제 대전 대덕구 지도(인구밀도: 교통량 = 5:5)에 대입하여 시각화한 사진

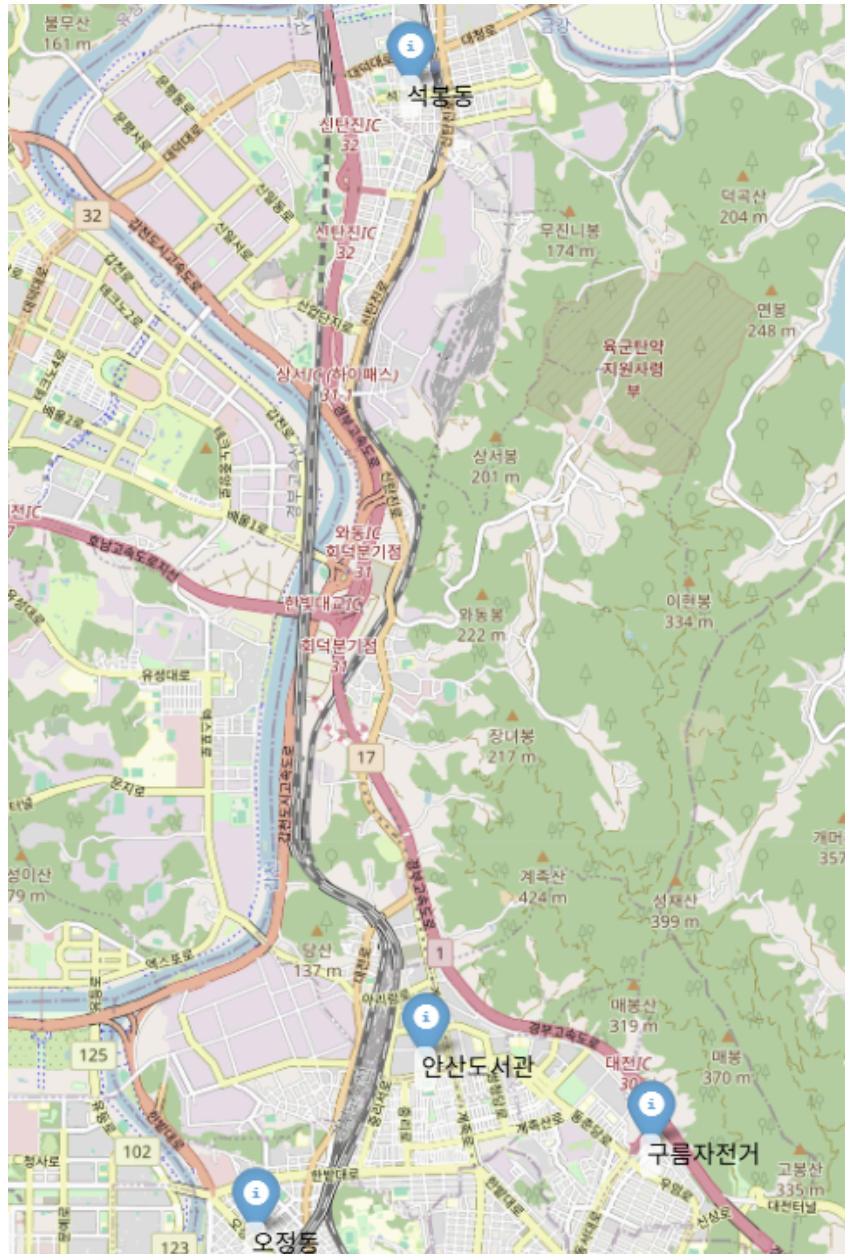
### 3. [대덕구] 인구 0.3 : 교통 (버스+지하철) 0.7

#가중치: 인구 0.3, 교통(버스+지하철) 0.7  
POP\_W = 0.3  
TRANSIT\_W = 0.7

## 결과



대덕구에 인구밀도 : 유동인구 비율을 3:7으로 적용한 결과



실제 대전 대덕구 지도(인구밀도: 교통량 = 3:7)에 대입하여 시각화한 사진

## 결론

서구와 대덕구를 분석한 결과, 인구밀도에 중점을 둘 경우와 교통량에 중점을 둘 경우 각각의 중요도 설정에 따라 각각 최적의 입지가 도출 되는 것을 알 수 있었다. 이는 정책 방향이나 설치 목적에 따라 가중치를 조정함으로써 다양한 의사결정 시나리오를 제시할 수 있음을 알 수 있다.